

GUÍA DE TESTING — NIVEL 1: RLS

SEGURIDAD EN SUPABASE

Row Level Security

PASO 0 — Preparación del Entorno de Testing

0.1 — Pre-requisitos

Antes de iniciar los tests, asegurate de haber ejecutado en el SQL Editor:

1. Todo el SQL de creación de tablas (Paso 0 del módulo)
2. Todas las políticas RLS del Nivel 1 (secciones 1.1, 1.2 y 1.3)
3. Los triggers (proteger_campos_sensibles, actualizar_updated_at)
4. Las funciones security definer (mi_rol, es_admin, es_docente, etc.)

0.2 — Crear Usuarios de Prueba

Registra 3 usuarios con roles diferentes usando la API de Auth de Supabase. Puedes hacerlo desde el Dashboard (Authentication → Users → Add User) o desde Postman/Insomnia:

Opción A: Desde Postman/Insomnia

Repite este request 3 veces cambiando email, nombre y rol:

```
POST {{SUPABASE_URL}}/auth/v1/signup

Headers:
  apikey: {{ANON_KEY}}
  -Type Content: application/json

Body:
{
  "email": "admin@test.upds.edu.bo",
  "password": "TestAdmin123!",
  "data": {
    "nombre_completo": "Admin upds Test",
    "rol": "admin"
  }
}
```

Usuarios a crear:

admin@test.upds.edu.bo / TestAdmin123! / rol: "admin"
docente@test.upds.edu.bo / TestDocente123! / rol: "docente"
alumno@test.upds.edu.bo / TestAlumno123! / rol: "estudiante"
[Alumno2@test.upds.edu.bo](#) / TestAlumno123! / rol: "estudiante"

Opción B: Desde el SQL Editor (forzar perfiles)

Si ya tienes usuarios creados en `auth.users`, puedes insertar los perfiles manualmente como superusuario:

```
-- Reemplaza los UUIDs con los reales de auth.users
INSERT INTO public.usuarios_perfil (id, nombre_completo, rol) VALUES
  ('UUID-ADMIN', 'Admin upds Test', 'admin'),
  ('UUID-DOCENTE', 'Docente upds Test', 'docente'),
  ('UUID-ALUMNO', 'Alumno upds Test', 'estudiante');

-- Crear una materia asignada al docente
INSERT INTO public.materias (id, codigo, nombre, creditos, docente_id, semestre)
VALUES (
  gen_random_uuid(),
  'SIS-301', 'Seguridad Informática', 4,
  'UUID-DOCENTE',
  '2026-I'
);

-- Inscribir al estudiante en esa materia
INSERT INTO public.inscripciones (estudiante_id, materia_id)
SELECT 'UUID-ALUMNO', id FROM public.materias WHERE codigo = 'SIS-301';
```

0.3 — Obtener Tokens JWT

Para probar con Postman/Insomnia necesitas el JWT de cada usuario. Click en login:

```
POST {{SUPABASE_URL}}/auth/v1/token?grant_type=password

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{
  "email": "admin@test.upds.edu.bo",
  "password": "TestAdmin123!"
}
```

La respuesta incluye el campo **access_token**. Copiá ese valor y guardálo como variable de entorno en Postman/Insomnia:

Variables de entorno recomendadas:

[illegible]

```
MWQILCJpc19hbm9ueW1vdXMiOmZhbHNlfQ.dvW61d1Og7LBdTOFXVmx9UAbkRtKEHCwNMa
5ZBm1AG1HXmm_sTCAEUjDWDyLRUKo3s4WHANPr_T3G0fbhfYgTA"
```

```
{{JWT_DOCENTE}} →
```

```
"eyJhbGciOiJFUzI1NiIsImtpZCI6ImVlZTQ3NmU1LTlwMDAtNGRhYy05ZDUzLTAYZmExNTNjNGRjNyIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwczovL2pkdHVobHfZ3FtYWVpamRoeXdpLnN1cGFYXNlLnVlZD1dGgvdjEiLCJzdWIiOiI2YjVjNzQ3OS0xNjc1LTQxMWEtOWVjNC00ZWVjZmZDg1YzUiLCJhdWQiOiJhdXRoZW50aWNhdGVkIiwiaXhwIjoxNzc2MzAyMjA1LCJpYXQiOiE3NzYyOTg2MDUsImVtYWlsIjoieG9jZW50ZUB0ZXN0LnVwZHMuzWR1LmJvIiwicGhvbmUiOiIiLCJhcHBfbWV0YWRhdGEiOnciOiJvdmlkZXIiOiJlbWVpbiCisInByb3ZpZGVycyl6WyJlbWVpbiCjdfSfidXNlcl9tZXRhZGF0YSI6eyJlbWVpbiF92ZXJpZmllZCI6dHJ1ZX0sInJvbiGUoiJhdXRoZW50aWNhdGVkIiwiaXhwIjoxNzc2MzAyMjA1LCJpYXQiOiE3NzYyOTg2MDV9XSwic2Vzcl9pZCI6IjU0ZTZmNjAzLTZhNmYtNDk4Yi04NzlhLWExNjVlNjAyMDhlYSIsImVzX2Fub255bW91cyI6ZmFsc2V9.W8t3lOIeGgceNxaXbn816ZcoGtinNfd265Ucla0aMSfe9fwUfjerNqkIV_3XQoSZXszd2KiCQZ4FBOCx2CiaQ"
```

```
{{JWT_ALUMNO}} → access_token del estudiante
```

```
{{SUPABASE_URL}} → https://zilayahylskwwmxvfhnrb.supabase.co
```

```
{{ANON_KEY}} → tu anon key (Settings → API)
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InppbGFoeWxza3d3bXh2ZmhuZmJlIiwiaXN0aWV0IjoxNzc2MzAyMjA1LCJpYXQiOiE3NzYyMTUyMzQsImVzX2Fub255bW91cyI6ZmFsc2V9.W8t3lOIeGgceNxaXbn816ZcoGtinNfd265Ucla0aMSfe9fwUfjerNqkIV_3XQoSZXszd2KiCQZ4FBOCx2CiaQ
```

0.4 — Estructura de requests en Postman/Insomnia

Todos los requests a la API REST de Supabase siguen este patrón:

```
GET/POST/PATCH {{SUPABASE_URL}}/rest/v1/nombre_tabla
```

Headers (siempre presentes):

```
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_DEL_USUARIO}}
Content-Type: application/json
```

Query params opcionales:

```
select=columna1,columna2
columna=eq.valor          (filtro WHERE)
order=columna.desc
```

Para INSERT: Prefer: return=representation

Para UPDATE: Prefer: return=representation

1.1 — Tests RLS Básico

Estos tests verifican las políticas fundamentales sobre **usuarios_perfil**: lectura, inserción y actualización del perfil propio, más el trigger de protección de campos.

Test 1.1.1 — Estudiante lee su propio perfil

Política verificada: **usuarios_select_propio** / **usuarios_select_propio_v2**

SQL Editor:

```
-- Ejecutar como el estudiante (usar su JWT en la sesión)
-- Desde la API REST o el cliente, esta consulta retorna solo su fila
SELECT id, nombre_completo, rol, carrera, activo
FROM public.usuarios_perfil;
```

Postman/Insomnia:

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol,carrera,activo
```

Headers:

apikey: {{ANON_KEY}}

Authorization: Bearer {{JWT_ALUMNO}}

X

Acción / Test	Rol	Resultado esperado	☑/✗
Estudiante consulta usuarios_perfil	Estudiante	Retorna solo SU perfil (1 fila)	☑
Verificar que no ve perfiles ajenos	Estudiante	0 filas de otros usuarios	☑

Test 1.1.2 — Estudiante NO puede ver perfiles ajenos

Política verificada: **usuarios_select_propio** (USING filtra por auth.uid())

Postman/Insomnia:

```
-- Intentar filtrar por el ID del docente usando el JWT del estudiante
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?id=eq.{{UUID_DOCENTE}}
```

Headers:

apikey: {{ANON_KEY}}

Authorization: Bearer {{JWT_ALUMNO}}

Acción / Test	Rol	Resultado esperado	☑/✗
Estudiante busca perfil del docente por ID	Estudiante	Retorna array vacío []	☑
Estudiante busca perfil del admin por ID	Estudiante	Retorna array vacío []	☑

Test 1.1.3 — Estudiante actualiza su propio perfil (campos permitidos)

Política verificada: **usuarios_update_propio**

Postman/Insomnia:

```
PATCH {{SUPABASE_URL}}/rest/v1/usuarios_perfil?id=eq.{{UUID_ALUMNO}}
```

Headers:

```
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_ALUMNO}}
Content-Type: application/json
Prefer: return=representation
```

Body:

```
{
  "nombre_completo": "Alumno upds Actualizado",
  "carrera": "Ingeniería de Sistemas"
}
```

Acción / Test	Rol	Resultado esperado	☑/✕
Estudiante cambia su nombre y carrera	Estudiante	UPDATE exitoso, campos actualizados	☑
Verificar updated_at se actualizó	Estudiante	updated_at tiene la fecha/hora actual	☑

Test 1.1.4 — Estudiante NO puede actualizar perfil ajeno

Política verificada: **usuarios_update_propio** (USING + WITH CHECK filtran por auth.uid())

Postman/Insomnia:

```
-- Intentar modificar el perfil del docente con el JWT del estudiante
PATCH {{SUPABASE_URL}}/rest/v1/usuarios_perfil?id=eq.{{UUID_DOCENTE}}
```

Headers:

```
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_ALUMNO}}
Content-Type: application/json
Prefer: return=representation
```

Body:

```
{
  "nombre_completo": "HACKEADO"
}
```

Acción / Test	Rol	Resultado esperado	☑/✕
Estudiante intenta editar perfil del docente	Estudiante	0 filas afectadas (silenciosamente ignorado)	☑
Verificar que el perfil del docente no cambió	Docente	nombre_completo intacto	☑

Test 1.1.5 — Trigger: Estudiante intenta cambiar su rol a admin

Mecanismo verificado: **trigger proteger_campos_sensibles** (defensa en profundidad)

Postman/Insomnia:

```
PATCH {{SUPABASE_URL}}/rest/v1/usuarios_perfil?id=eq.{{UUID_ALUMNO}}
```

Headers:

```
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_ALUMNO}}
Content-Type: application/json
Prefer: return=representation
```

Body:

```
{
  "rol": "admin",
  "nombre_completo": "Intento de Escalación"
}
```

SQL Editor (verificación):

```
-- Ejecutar como superusuario para verificar el resultado
SELECT id, nombre_completo, rol, activo, eliminado
FROM public.usuarios_perfil
WHERE id = 'UUID-ALUMNO';

-- Resultado esperado:
-- nombre_completo = 'Intento de Escalación' (sí cambió)
-- rol = 'estudiante' (NO cambió → trigger lo protegió)
```

Acción / Test	Rol	Resultado esperado	☑/✗
Estudiante envía rol: 'admin' en el PATCH	Estudiante	El request no da error (HTTP 200)	☑
Verificar que el campo rol NO cambió	Admin/SQL	rol sigue siendo 'estudiante'	☑
Verificar que nombre_completo SÍ cambió	Admin/SQL	nombre_completo actualizado	☑
Verificar que activo y eliminado no cambiaron	Admin/SQL	activo = true, eliminado = false	☑

Concepto clave — Defensa en profundidad:

Este es el test más importante del nivel básico. Demuestra que RLS permite el UPDATE en la fila (porque es su perfil), pero el trigger impide la escalación de privilegios. Dos capas trabajando juntas.

Test 1.1.6 — Request sin autenticación (rol anon)

Verificar que el rol **anon** no tiene acceso a usuarios_perfil (no hay política TO anon).

Postman/Insomnia:

```
-- Request SIN header Authorization (solo con la anon key)
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=*
```

Headers:

```
apikey: {{ANON_KEY}}
-- NO incluir Authorization
```

Acción / Test	Rol	Resultado esperado	☑/✕
Request sin JWT a usuarios_perfil	anon	Retorna array vacío [] o error 401	☑

1.2 — Tests RLS Intermedio

Estos tests verifican políticas con **subqueries, joins y relaciones entre tablas**: materias, inscripciones, calificaciones y documentos.

Test 1.2.1 — Materias: Lectura de materias activas

Política verificada: **materias_select_autenticados** (activa = true)

Postman/Insomnia:

```
GET {{SUPABASE_URL}}/rest/v1/materias?select=id,codigo,nombre,semestre
```

Headers:

apikey: {{ANON_KEY}}

Authorization: Bearer {{JWT_ALUMNO}}

SQL Editor (preparar materia inactiva para contrastar):

```
-- Crear una materia inactiva para verificar que NO aparece
INSERT INTO public.materias (codigo, nombre, creditos, docente_id, semestre, activa)
VALUES ('TST-999', 'Materia Oculta Test', 3, 'UUID-DOCENTE', '2025-II', false);
```

Acción / Test	Rol	Resultado esperado	☑/✗
Estudiante lista materias	Estudiante	Ve solo materias con activa = true	☑
Verificar que TST-999 NO aparece	Estudiante	La materia inactiva es invisible	☑
Docente lista materias	Docente	También ve solo materias activas	☑

Test 1.2.2 — Materias: Docente edita SU materia vs materia ajena

Política verificada: **materias_update_docente** (docente_id = auth.uid())

Postman/Insomnia:

```
-- Test A: Docente edita SU materia (debe funcionar)
PATCH {{SUPABASE_URL}}/rest/v1/materias?codigo=eq.SIS-301
```

Headers:

apikey: {{ANON_KEY}}

Authorization: Bearer {{JWT_DOCENTE}}

Content-Type: application/json

Prefer: return=representation

Body:

```
{ "nombre": "Seguridad Informática Avanzada" }
```

SQL Editor (preparar segunda materia de otro docente):

```
-- Crear materia de OTRO docente (el admin en este caso)
INSERT INTO public.materias (codigo, nombre, creditos, docente_id, semestre)
VALUES ('MAT-101', 'Cálculo I', 5, 'UUID-ADMIN', '2026-I');
```

```
-- Test B: Docente intenta editar materia AJENA (debe fallar)
PATCH {{SUPABASE_URL}}/rest/v1/materias?codigo=eq.MAT-101
```


Headers:
 apikey: {{ANON_KEY}}
 Authorization: Bearer {{JWT_DOCENTE}}
 Content-Type: application/json
 Prefer: return=representation

Body:
 { "nombre": "MATERIA HACKEADA" }

Acción / Test	Rol	Resultado esperado	☑/✕
Docente edita SU materia (SIS-301)	Docente	UPDATE exitoso, nombre actualizado	☑
Docente edita materia AJENA (MAT-101)	Docente	0 filas afectadas, no modifica nada	☑

Test 1.2.3 — Inscripciones: Estudiante vs Docente

Políticas verificadas: inscripciones_select_estudiante + inscripciones_select_docente

Postman/Insomnia:

```
-- Como estudiante: ver MIS inscripciones
GET {{SUPABASE_URL}}/rest/v1/inscripciones?select=id,materia_id,estado
```

Headers:
 apikey: {{ANON_KEY}}
 Authorization: Bearer {{JWT_ALUMNO}}

```
-- Como docente: ver inscripciones de MIS materias
GET {{SUPABASE_URL}}/rest/v1/inscripciones?select=id,estudiante_id,materia_id,estado
```

Headers:
 apikey: {{ANON_KEY}}
 Authorization: Bearer {{JWT_DOCENTE}}

Acción / Test	Rol	Resultado esperado	☑/✕
Estudiante lista inscripciones	Estudiante	Ve solo SUS inscripciones	☑
Docente lista inscripciones	Docente	Ve inscripciones de SUS materias	☑
Docente NO ve inscripciones de MAT-101	Docente	Si MAT-101 no es suya, no aparecen	☑

Test 1.2.4 — Calificaciones: CRUD del docentePolíticas verificadas: **calificaciones_select/insert/update_docente****Paso A: Docente inserta una nota en SU materia**

POST {{SUPABASE_URL}}/rest/v1/calificaciones

Headers:

```
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_DOCENTE}}
Content-Type: application/json
Prefer: return=representation
```

Body:

```
{
  "inscripcion_id": "{{ID_INSCRIPCION_ALUMNO_EN_SIS301}}",
  "tipo_evaluacion": "parcial_1",
  "nota": 78.5,
  "registrado_por": "{{UUID_DOCENTE}}"
}
```

Paso B: Docente actualiza la nota que registró

PATCH {{SUPABASE_URL}}/rest/v1/calificaciones?id=eq.{{ID_CALIFICACION_CREADA}}

Headers:

```
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_DOCENTE}}
Content-Type: application/json
Prefer: return=representation
```

Body:

```
{ "nota": 82.0, "observaciones": "Corregido por revisión" }
```

Paso C: Estudiante lee SUS notas

GET {{SUPABASE_URL}}/rest/v1/calificaciones?select=tipo_evaluacion,nota,fecha,observaciones

Headers:

```
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_ALUMNO}}
```

Acción / Test	Rol	Resultado esperado	☑/✗
Docente INSERT nota en SU materia	Docente	INSERT exitoso, retorna la fila	☑
Docente UPDATE nota que él registró	Docente	UPDATE exitoso, nota cambia a 82.0	☑
Estudiante lee sus notas	Estudiante	Ve la calificación con nota 82.0	☑
Estudiante NO ve notas de otros	Estudiante	Solo ve notas de sus inscripciones	☑

Test 1.2.5 — Calificaciones: Docente NO puede insertar en materia ajenaPolítica verificada: **calificaciones_insert_docente** (WITH CHECK con subquery)**SQL Editor (preparar datos):**

```
-- Inscribir al alumno en la materia de OTRO docente (MAT-101)
INSERT INTO public.inscripciones (estudiante_id, materia_id)
SELECT 'UUID-ALUMNO', id FROM public.materias WHERE codigo = 'MAT-101';
```

Postman/Insomnia:

```
-- Docente intenta poner nota en materia AJENA (MAT-101)
POST {{SUPABASE_URL}}/rest/v1/calificaciones

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_DOCENTE}}
  Content-Type: application/json
  Prefer: return=representation

Body:
{
  "inscripcion_id": "{{ID_INSCRIPCION_ALUMNO_EN_MAT101}}",
  "tipo_evaluacion": "parcial_1",
  "nota": 99,
  "registrado_por": "{{UUID_DOCENTE}}"
}
```

Acción / Test	Rol	Resultado esperado	☒ /✕
Docente INSERT en materia AJENA	Docente	Error 403: new row violates RLS policy	✕

Test 1.2.6 — Documentos: Visibilidad controladaPolíticas verificadas: **documentos_select/insert_docente + documentos_select_estudiante****SQL Editor (preparar documentos):**

```
-- Docente sube un documento visible
INSERT INTO public.documentos_academicos
(titulo, ruta_archivo, tipo_documento, materia_id, subido_por, visible_estudiantes)
SELECT
'Syllabus Seguridad', '/docs/syllabus.pdf', 'syllabus',
m.id, 'UUID-DOCENTE', true
FROM public.materias m WHERE m.codigo = 'SIS-301';

-- Docente sube un documento OCULTO (examen)
INSERT INTO public.documentos_academicos
(titulo, ruta_archivo, tipo_documento, materia_id, subido_por, visible_estudiantes)
SELECT
'Examen Final', '/docs/examen.pdf', 'examen',
m.id, 'UUID-DOCENTE', false
FROM public.materias m WHERE m.codigo = 'SIS-301';
```

Postman/Insomnia:

```
-- Estudiante consulta documentos
GET
{{SUPABASE_URL}}/rest/v1/documentos_academicos?select=titulo,tipo_documento,visible_estudiantes

Headers:
apikey: {{ANON_KEY}}
Authorization: Bearer {{JWT_ALUMNO}}
```

Acción / Test	Rol	Resultado esperado	☑/✗
Estudiante lista documentos	Estudiante	Ve solo 'Syllabus' (visible = true)	☑
Estudiante NO ve 'Examen Final'	Estudiante	Documento con visible = false oculto	☑
Docente lista documentos	Docente	Ve ambos documentos de SU materia	☑

1.3 — Tests RLS Avanzado

Estos tests verifican **funciones security definer, políticas de admin, soft delete y vistas seguras**.

Test 1.3.1 — Funciones Security Definer

Funciones verificadas: `mi_rol()`, `es_admin()`, `es_docente()`, `es_docente_de_materia()`

Postman/Insomnia (llamar funciones RPC):

```
-- Test A: mi_rol() como estudiante
POST {{SUPABASE_URL}}/rest/v1/rpc/mi_rol

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
  Content-Type: application/json

Body: {}
```

```
-- Test B: es_admin() como admin
POST {{SUPABASE_URL}}/rest/v1/rpc/es_admin

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ADMIN}}
  Content-Type: application/json

Body: {}
```

```
-- Test C: es_docente_de_materia() con UUID de SIS-301
POST {{SUPABASE_URL}}/rest/v1/rpc/es_docente_de_materia

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_DOCENTE}}
  Content-Type: application/json

Body:
{ "p_materia_id": "{{UUID_MATERIA_SIS301}}" }
```

Acción / Test	Rol	Resultado esperado	☑/✕
<code>mi_rol()</code> como estudiante	Estudiante	Retorna "estudiante"	☑
<code>mi_rol()</code> como docente	Docente	Retorna "docente"	☑
<code>es_admin()</code> como admin	Admin	Retorna true	☑
<code>es_admin()</code> como estudiante	Estudiante	Retorna false	☑
<code>es_docente_de_materia(SIS-301)</code> como docente	Docente	Retorna true (es su materia)	☑
<code>es_docente_de_materia(MAT-101)</code> como docente	Docente	Retorna false (no es su materia)	☑

Test 1.3.2 — Admin: Acceso total a todas las tablas

Políticas verificadas: `*_all_admin` (FOR ALL en cada tabla)

Postman/Insomnia:

```
-- Admin lee TODOS los perfiles
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol,activo,eliminado
```

```
Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ADMIN}}
```

```
-- Admin lee TODAS las calificaciones
GET {{SUPABASE_URL}}/rest/v1/calificaciones?select=*
```

```
Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ADMIN}}
```

Acción / Test	Rol	Resultado esperado	☑/✕
Admin lista todos los perfiles	Admin	Ve los 3 usuarios (admin, docente, alumno)	☑
Admin lista todas las inscripciones	Admin	Ve todas las inscripciones del sistema	☑
Admin lista todas las calificaciones	Admin	Ve todas las notas registradas	☑
Admin lista todos los documentos	Admin	Ve todos (visibles y ocultos)	☑

Test 1.3.3 — Soft Delete: Ejecución y verificación de visibilidad

Mecanismos verificados: **soft_delete_usuario()** + **usuarios_select propio_v2** + **usuarios_select_admin**

Paso A: Admin ejecuta soft delete del estudiante

```
-- Vía RPC desde Postman/Insomnia
POST {{SUPABASE_URL}}/rest/v1/rpc/soft_delete_usuario

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ADMIN}}
  Content-Type: application/json

Body:
{ "p_usuario_id": "{{UUID_ALUMNO}}" }
```

Paso B: Verificar desde SQL Editor

```
-- Como superusuario, verificar el estado del estudiante
SELECT id, nombre_completo, rol, activo, eliminado
FROM public.usuarios_perfil
WHERE id = 'UUID-ALUMNO';

-- Esperado: activo = false, eliminado = true
```

Paso C: Estudiante intenta leer su perfil

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=*

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
```

Paso D: Admin verifica que aún ve al estudiante eliminado

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol,eliminado

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ADMIN}}
```

Acción / Test	Rol	Resultado esperado	☑/✗
Admin ejecuta soft_delete_usuario()	Admin	Ejecución exitosa (HTTP 200/204)	☑
Verificar estado en DB	SQL Editor	eliminado = true, activo = false	☑
Estudiante consulta su perfil	Estudiante	Retorna array vacío [] (invisible)	☑
Admin consulta perfiles	Admin	Ve al estudiante con eliminado = true	☑
Estudiante NO-admin intenta soft_delete	Estudiante	Error: Solo administradores pueden...	✗

Paso E: Restaurar al estudiante para futuros tests

```
-- Desde SQL Editor como superusuario
UPDATE public.usuarios_perfil
SET eliminado = false, activo = true
WHERE id = 'UUID-ALUMNO';
```

Test 1.3.4 — Vista segura: vista_notas con security_invoker

Mecanismo verificado: **vista_notas con security_invoker = true** (respeto RLS del usuario que invoca)

Postman/Insomnia:

```
-- Estudiante consulta la vista de notas
GET {{SUPABASE_URL}}/rest/v1/vista_notas?select=estudiante,materia,tipo_evaluacion,nota

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
```

```
-- Docente consulta la vista de notas
GET {{SUPABASE_URL}}/rest/v1/vista_notas?select=estudiante,materia,tipo_evaluacion,nota

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_DOCENTE}}
```

Acción / Test	Rol	Resultado esperado	☑/✕
Estudiante consulta vista_notas	Estudiante	Ve solo SUS notas (filtrado por RLS)	☑
Docente consulta vista_notas	Docente	Ve notas de SUS materias	☑
Admin consulta vista_notas	Admin	Ve todas las notas del sistema	☑

Checklist de Verificación — Nivel 1 Completo

Marcá cada test a medida que lo completés:

☒	Test	Descripción	OK
☐	1.1.1	Estudiante lee su propio perfil	
☐	1.1.2	Estudiante NO ve perfiles ajenos	
☐	1.1.3	Estudiante actualiza campos permitidos de su perfil	
☐	1.1.4	Estudiante NO puede actualizar perfil ajeno	
☐	1.1.5	Trigger bloquea cambio de rol (defensa en profundidad)	
☐	1.1.6	Request sin autenticación (anon) no ve datos	
☐	1.2.1	Materias: solo visibles las activas	
☐	1.2.2	Docente edita SU materia / NO puede editar ajena	
☐	1.2.3	Inscripciones filtradas por estudiante y docente	
☐	1.2.4	Calificaciones: INSERT, UPDATE y SELECT por docente	
☐	1.2.5	Docente NO puede insertar en materia ajena	
☐	1.2.6	Documentos: visibilidad controlada por visible_estudiantes	
☐	1.3.1	Funciones security definer retornan valores correctos	
☐	1.3.2	Admin tiene acceso total a todas las tablas	
☐	1.3.3	Soft delete: oculta para usuario, visible para admin	
☐	1.3.4	Vista vista_notas respeta RLS (security_invoker)	